SOFTWARE LAB 10

ARYAN RAJ - 24BDS1094

YASH RAJ - 24BDS1209

AIM

Develop separate Component and Deployment Diagrams for the YieldnYarn project using StarUML.

YieldnYarn is a marketplace connecting MSME sellers and buyers of agricultural and textile commodities. It supports quality grading, market price estimation, and order matching.

1 Description of Components

1. Web Application: Provides the buyer, administrator, and verifier interfaces using Next.js. Users can browse listings, submit requirements, manage orders, and review quality grades.

2. Seller Application: Provides an Android interface using Kotlin. Sellers can create listings, upload evidence photographs, check prices, and synchronise pending changes after reconnecting.

3. Offline Data Store: Stores local listing data in a Room SQLite database. WorkManager schedules synchronisation when the seller device has a network connection.

4. Marketplace Service: Uses Django and Django REST Framework for authentication, roles, catalog management, verification, requirements, orders, bids, disputes, reputation, notifications, and vertical configuration.

5. Compute Service: Uses FastAPI for AI quality grading, pricing intelligence, matching and allocation. Its scheduler also collects external market price information.

6. Marketplace Persistence: Stores users, listings, grading results, price data, requirements, bids, orders, and disputes in PostgreSQL. Django and FastAPI access the shared database.

7. Evidence Storage: Stores photographs submitted with listings. Django uploads the evidence, and the compute service reads the photographs during quality grading.

8. Agmarknet Price Feed: Provides commodity market prices through the data.gov.in API. The compute service collects these prices for market price estimation and trends.

9. Firebase Messaging: Provides optional push notifications to the seller application. This integration requires Firebase credentials and device configuration.

2 Relationships Between Components

1. Applications and Marketplace API: The web and seller applications depend on the Marketplace REST API provided by the Django service for authentication and marketplace operations.

2. Seller Application and Offline Data Store: The seller application saves local data and synchronises pending listings and evidence when connectivity is available.

3. Marketplace Service and Compute API: Django requests quality grading and matching from FastAPI. These requests include an internal token for service authentication.

4. Applications and Public Pricing API: The web and seller applications can directly request read-only price estimates and trends from FastAPI.

5. Backend Services and Storage: Both backend services access PostgreSQL. Django stores evidence photographs, while FastAPI reads them for grading.

6. External Services: FastAPI obtains market prices from Agmarknet. Django can request optional Firebase push delivery to registered seller devices.

The Component Diagram uses component symbols, rectangular interface symbols, dependency arrows, and interface realisation relationships. Nested modules belong to their enclosing service.

3 Description of Deployment Nodes

The Deployment Diagram represents the Azure deployment design configured for YieldnYarn. It shows the software placed on each device or execution environment.

1. User Computer: Runs a web browser that loads the web interface and sends requests to the backend APIs.

2. Seller Smartphone: Runs the Android application and its local SQLite database. The application is installed as an APK.

3. Azure Container Apps Environment: Hosts the web, Django, and FastAPI containers within a common managed environment.

4. Web Container: Runs the Next.js standalone application using Node.js on application port 3000.

5. Django Container: Runs the marketplace API using Gunicorn on application port 8000.

6. FastAPI Container: Runs the compute API using Uvicorn on application port 8001. Its image includes the grading models, and one replica runs the scheduler.

7. Azure PostgreSQL Server: Hosts the shared msme_marketplace database using PostgreSQL 16.

8. Azure Blob Storage and External Services: Blob Storage holds private evidence photographs. The data.gov.in API and optional Firebase service are external nodes.

4 Communication Between Deployment Nodes

1. Browser to Web Container: The browser loads the web application over HTTPS. Azure receives the connection on port 443 and routes it to application port 3000.

2. Clients to Django Container: The browser and Android application send REST requests over HTTPS on port 443. Azure routes these requests to Django on application port 8000.

3. Clients to FastAPI Container: Both clients can access public pricing endpoints over HTTPS. Azure routes the requests to application port 8001.

4. Django to FastAPI: Django sends grading and matching requests over HTTPS with the X-Internal-Token header. FastAPI checks this token before processing protected requests.

5. Backend Containers to Database: Django and FastAPI communicate with PostgreSQL over TLS on port 5432. Django uses its ORM, and FastAPI uses SQLAlchemy.

6. Backend Containers to Blob Storage: Django uploads evidence using HTTPS. FastAPI reads evidence photographs using a read-only shared access signature, or SAS.

7. FastAPI to Market Price API: The compute service retrieves Agmarknet market prices from data.gov.in using HTTPS and stores the processed price data in PostgreSQL.

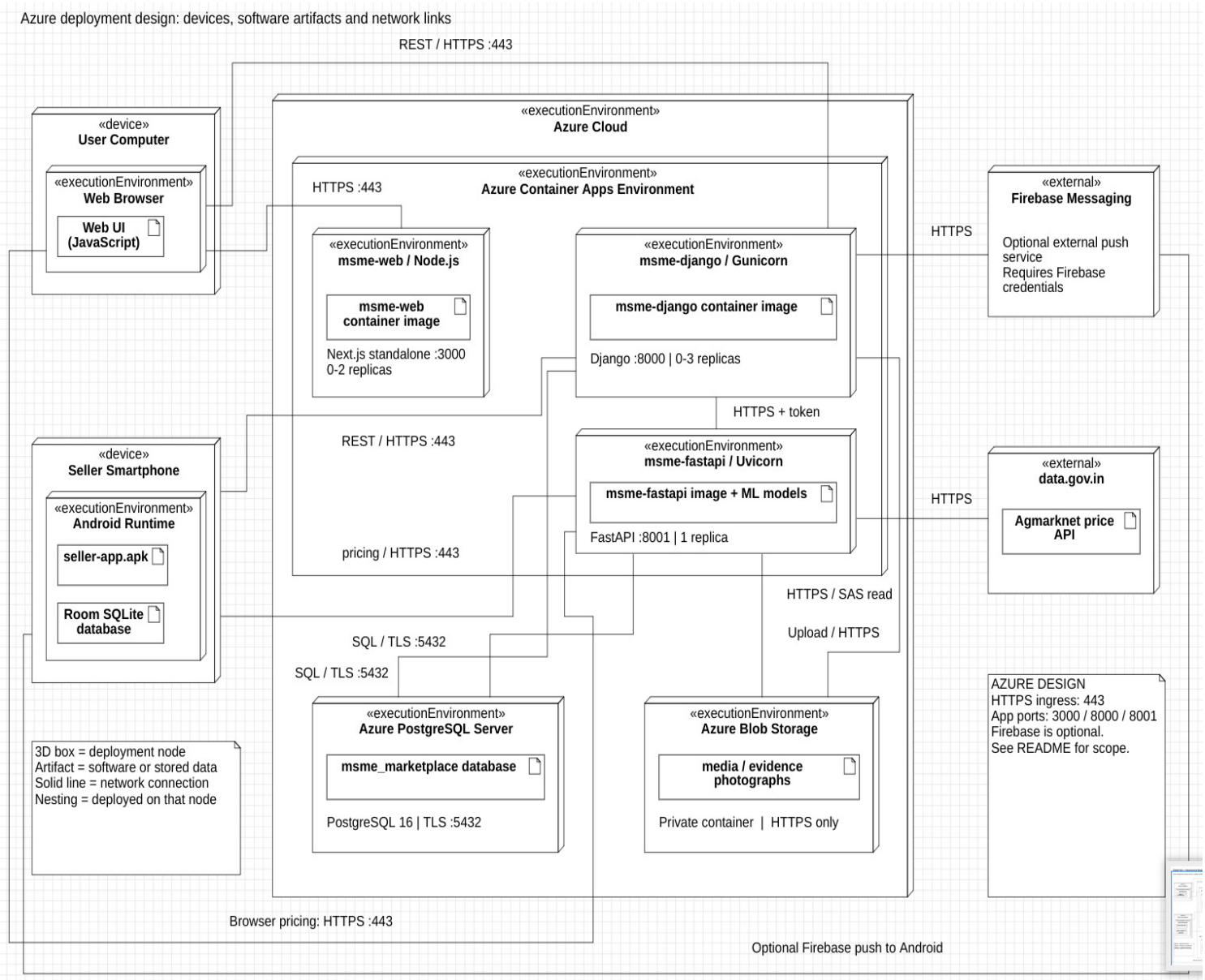
8. Optional Firebase Communication: When configured, Django sends push requests to Firebase, which delivers notifications to the Android application. Firebase is configured separately from the Azure deployment template.

The Deployment Diagram uses three-dimensional node symbols, artifact symbols, and solid communication paths. Artifacts such as container images, the APK, databases, and photographs are placed inside the nodes that run or store them.

5 Result

Separate Component and Deployment Diagrams were developed for YieldnYarn in StarUML. The Component Diagram describes the software modules and their dependencies. The Deployment Diagram describes the Azure deployment design, deployed artifacts, and communication between nodes.

7 Component Diagram



8 Deployment Diagram

